GON AI DEVELOPMENT OS · ORCA EDITION 1.0

어떤 창에서, 무엇을 입력하고, 어디에 남길까요?

Orca + Codex CLI로 아이디어를 조사하고, 작업자를 조정하고,
GitHub에 연결해 실제 개발까지 이어가는 실행 안내서.

P1~P5 전체 Codex CLI | 세션 유지·재개·새 시작 | 문서 → 작업자 → PR

확인 기준 2026-11-02 · Orca CLI 1.4.246 · Codex CLI 0.166.0
설치 가이드·도움말과 공식 문서를 대조한 교육용 안내서입니다.

먼저 2분

이 안내서는 이렇게 사용하세요

1. 아래 프로젝트명과 폴더를 내 값으로 바꿉니다. 아직 GitHub 이름을 정하지 않았다면 비워 두고 G1 전에 입력합니다.
2. 흐름도에서 **S0**를 누릅니다. 단계의 "열 창 → 순서 → 요청문 → 확인"을 따라갑니다.
3. 자연어 요청문은 **Codex 입력칸**에, 명령은 **일반 터미널**에 붙여 넣습니다. 복사 버튼은 명령을 실행하지 않습니다.

프로젝트명

내 프로젝트

프로젝트 폴더 · 절대 경로

/home/gon/dev/apps/my-project

GitHub 소유자/저장소 · G1 전 입력

예: my-account/my-project

처음 보이는 폴더는 예시입니다. 실제 폴더를 만든 뒤 그 경로로 바꾸세요. 설정·확인 체크는 현재 브라우저에 저장됩니다. 다른 기기나 PDF에는 자동 동기화되지 않습니다.

요청문에 반영했습니다. GitHub 이름이 없는 요청은 G1 전에 입력하세요.

전체 흐름 · 단계를 누르면 상세 설명이 열립니다

기획은 같은 대화, 개발은 문서를 읽는 새 대화



P4 선택 근거 부족 → P3 문제가 달라짐 → P1 기존 도구 사용·보류 → 결정과 STATE 저장 후 종료

G1까지는 같은 기획 주 조정자가 마무리합니다. 활성 조사 작업과 문서 저장을 정리한 뒤 D1에서 새 개발 세션을 엽니다.

화면 배치

사용자는 주 조정자와 대화하고, 주 조정자가 작업자를 관리합니다



설명용 배치도이며 실제 Orca 화면 캡처가 아닙니다. 탭·분할 방식은 기존 사용자 설정을 따릅니다.

기획할 때

주 조정자와 조사 작업자가 같은 폴더를 쓸 수 있습니다. 결과 파일을 다르게 지정하고 통합 문서는 주 조정자만 수정합니다.

코드를 병렬로 고칠 때

충돌 가능한 작업은 Orca가 만든 별도 worktree에 배치합니다. 작업자는 자신의 커밋과 PR을 보고하고 주 조정자가 결과를 통합합니다.

CLI 모드 확인: 주 조정자는 agent launch의 `--surface terminal` 로 CLI를 선택할 수 있습니다. worker-start에는 같은 옵션이 없으며 사용자의 새 에이전트 Chat | CLI 설정을 따릅니다. CLI를 원하면 Settings → Agents → Choose per agent에서 Codex를 Terminal로 맞추고 시작 결과를 확인하세요. 다른 형태로 시작되어도 조정은 같은 orchestration 명령을 사용합니다.

세션 판단표

지금 대화를 이어갈까요, 새로 시작할까요?

상황	선택	실제로 할 일
P1 → P2 → P3 → P4 → P5	같은 기획 세션	단계마다 /new를 누르지 않음. 파일로 합의를 고정.
독립 조사·구현을 맡김	새 작업자 세션	주 조정자가 worker-start. 동일 대화를 동시에 resume하지 않음.
작업자가 일하는 중	주 조정자 유지	응답은 끝내고 Orca 알림을 기다림. 탭·CLI는 종료하지 않음.
내일 같은 일을 이어서 함	기존 세션 resume	같은 프로젝트·같은 Codex 설정 홈에서 이름/ID 확인.
P5·Git 기준 저장 → 개발	새 개발 조정 세션	최종 문서 3종을 읽고 시작. 활성 기획 작업은 먼저 정리.
첫 출시 안의 수정·검증	같은 개발 세션	작은 구현·검증을 반복하고 관련 작업자에게 send.
첫 출시 완료 → 독립 새 기능	새 세션·새 작업 묶음	최신 문서·이슈에서 시작. 이전 결과를 인계.
대안만 따로 탐색	필요하면 fork	대화 분기이며 Git 브랜치/worktree 분리와 다름.
대화가 길어짐	문서 저장 후 compact	맥락 요약일 뿐 파일·GitHub·Run 상태 저장을 대신하지 않음.

근거: Codex 공식 명령 설명 · resume 동작 · 설치 Orca orchestration 가이드. “P5 후 새 개발 세션”은 이 안내서의 운영 권장 방식입니다.

▼ 오늘 멈추고 내일 재개하는 정확한 순서

1. 현재 주 조정자에게 아래 “중단 전 저장” 요청을 보냅니다. 활성 작업자가 있으면 그대로 종료하지 말고 상태를 확인합니다.
2. 기록이 저장되고 종료 가능한 상태이면 현재 Codex에 `/quit`를 한 줄로 입력합니다.
3. 다음에 Orca에서 같은 작업공간·같은 실행 환경을 엽니다. 기존 Codex 탭이 살아 있다면 그 탭을 계속 씁니다.
4. 종료된 경우 일반 터미널에서 아래 resume 명령으로 세션 목록을 열고 프로젝트-기획 또는 프로젝트-개발 이름과 경로를 확인해 선택합니다. 작업자 대화를 잘못 선택하지 않습니다.
5. 재개 요청을 보냅니다. Codex 대화 복구와 Orca Run 연결은 별도입니다. 이전 조정자가 살아 있다면 같은 Run을 다른 조정자가 빼앗아 관리하지 않습니다.

현재 주 조정자 · 중단 전 저장

```
중단 전에 docs/STATE.md를 갱신해줘. 현재 단계, 최종 문서 버전, 실제 파일·미커밋 변경, 기준/현재 커밋, GitHub 이슈·PR, Run·Task·Dispatch, 활성 작업자, 남은 문제, 다음 행동을 적어.
아직 실행 중인 작업자가 있으면 현재 조정 세션을 종료해도 되는지 실제 상태와 설치 가이드 기준으로 알려줘. 종료를 확인하지 못하면 종료됐다고 쓰지 마.
저장한 파일을 다시 읽어 경로와 다음에 보낼 재개 요청을 알려줘.
```

현재 Codex 입력칸 · 한 줄

```
/quit
```

종료된 경우 · Orca 일반 터미널에서 목록 열기

```
cd '/home/gon/dev/apps/my-project'
codex resume --search
```

최근 세션 --last는 작업자와 혼동하기 쉬우므로 이름·경로로 선택합니다.

재개한 주 조정 Codex 입력칸

```
현재 docs/STATE.md와 최종 문서·TASKS를 읽고 실제 파일·Git 상태·미커밋 변경을 대조해줘.
STATE의 Orca Run/Task/Dispatch를 현재 상태와 비교하고, 이전 조정자가 살아 있는지 먼저 확인해.
이전 조정자가 종료된 것이 확인되고 내가 그 작업을 이어받는 경우에만 설치 가이드에 따라 기존 Run을 다시 연결해. 아직 실행 중인 작업자를 중복 시작하지 마.
완료·실패·미검증·다음 행동 하나를 요약한 뒤 같은 승인 범위의 작업을 이어가.
```

목록에 대화가 없으면 실행한 사용자·프로젝트 경로·Codex 설정 홈을 확인합니다. Orca가 별도 런타임이나 계정 환경을 썼다면 외부 터미널의 Codex 목록과 다를 수 있습니다. 인증 파일을 옮겨 해결하려 하지 않습니다.

▼ /new · /fork · /compact를 언제 쓰나요?

`/new`는 현재 CLI 안에서 새 대화를 시작합니다. `/fork`는 이전 대화 맥락을 가진 별도 대화로 분기합니다. 둘 다 Git 변경을 자동 분리하지 않습니다. `/compact`는 긴 대화를 요약합니다. 작업자가 진행 중인 주 조정자에서는 대화를 바꾸기 전에 조정 상태를 확인하세요.

기획에서 개발로 넘어갈 때는 새 탭과 명확한 이름을 써서 역할을 구분하는 편을 권합니다. fork는 별도 대안 검토용이며 필수 단계가 아닙니다.

문서 연결 · 모든 경로는 프로젝트의 docs/ 아래

리서치 4개 파일을 개발 기준 3종으로 정제합니다

내 말과 조사

P1 질문·답변
P2 Q번호·계획
P3 S/U 근거
P4 D번호·결정

→

P5 최종 문서 3종

PROJECT-BRIEF · 무엇을 / 완료 기준
TASKS · 순서 / 담당 / 검증
STATE · 현재 상태 / 다음 행동

→

개발과 GitHub

이슈에 R/AC 연결
Orca Task에 파일 책임
PR에 변경·실제 검증
새 세션은 3종부터 읽기

단계	저장 파일	무엇을 남기는가	누가 쓰는가
P1	research/01-problem.md	질문·내 답변·문제·제약·미정	주 조정자
P2	research/02-research-plan.md	Q번호·조사 방법·담당·종료 기준	주 조정자
P3 전	research/notes/*.md	작업자별 원문 조사와 한계	각 작업자 자기 파일
P3	research/03-evidence.md	Q에 답하는 S/U 근거·날짜·해석·빈틈	주 조정자 통합
P4	research/04-decision.md	D번호·선택·이유·근거·보류·재검토	주 조정자 + 내 실제 선택
P5	PROJECT-BRIEF.md	무엇을 만들고 무엇으로 확인할지	주 조정자
P5 → D2	TASKS.md	어떤 순서·담당 파일·R/AC로 실행할지	주 조정자
모든 단계	STATE.md	지금 어디까지 했고 다음에 무엇을 할지	주 조정자

복사·합치기의 뜻: 원본 4개를 지우고 큰 파일 하나로 만드는 것이 아닙니다. 최종 문서에는 선택된 결론과 실행 기준만 적고, "왜?"에 답할 때 원래 근거로 돌아갈 링크를 둡니다.

한 요구가 끝까지 이어지는 예

Q 질문	S/U 근거	D 결정	R 요구	AC 완료 기준	작업 / GitHub
Q02/03 저장 범위·완료 기준	U02 필요하다는 답변 · U03 범위 확인	D02 같은 브라우저 저장	R04 새로고침 유지	AC04 새로고침 후 항목·완료 상태 일치	K03 → 실제 이슈 → 구현 PR

설명용 가상 연결입니다. 실제 관찰·승인·GitHub 이슈를 만들었다는 뜻이 아닙니다. 저장 API의 한계가 미확인이면 구현 전에 확인하는 조건으로 유지합니다.

새 개발 세션에 전달하는 최소 묶음

```
프로젝트/
└── docs/
    ├── PROJECT-BRIEF.md  ← 무엇을 만들고 무엇으로 확인하나
    ├── TASKS.md          ← 순서·선행 관계·담당 파일
    ├── STATE.md          ← 지금 상태·다음 행동·GitHub·Orca ID
    ├── tasks/          ← 작업자별 지시서
    └── research/
        ├── 01-problem.md
        ├── 02-research-plan.md
        ├── 03-evidence.md
        ├── 04-decision.md
        └── notes/        ← 작업자별 조사 메모
```

S0

Orca에 프로젝트와 기획 Codex를 연다

프로젝트 폴더, 대화 입력칸, 명령 입력칸을 먼저 구분합니다.

열 창 Orca 앱 → 해당 프로젝트 작업공간 → Codex CLI 탭

세션 새 기획 세션 1개

1. 이 순서로 진행하세요

1. Orca의 Add a project(프로젝트 추가)를 엽니다. 새로 시작하면 Create new project → 프로젝트 이름과 Parent folder(상위 폴더) → Create project 순서로 만듭니다. 기존 폴더라면 Browse folder → 해당 폴더 선택을 사용합니다. Git이 없는 폴더에는 Open as Folder를 선택해 기존 상태를 유지할 수 있습니다.
2. 기존 GitHub 저장소에서 시작한다면 Add a project → Clone from URL → GitHub 저장소 URL과 Parent folder → Clone을 선택합니다. 이미 복제해 둔 폴더가 있다면 다시 복제하지 말고 Browse folder로 엽니다. 새로 만든 Orca 프로젝트는 로컬 Git을 준비할 수 있지만 GitHub 원격 업로드는 G1에서 별도로 합니다.
3. 아래 terminal create 명령으로 현재 작업공간에 codex --yolo를 실행합니다. 명령은 셸 입력칸에, 다음 자연어 요청은 열린 Codex 입력칸에 넣습니다.
4. 주 조정자 탭은 “기획”, 별도 일반 터미널은 “명령·확인” 용도로 씁니다. 이는 이 안내서의 역할 이름입니다. 일반 터미널은 파일 확인·Git 명령용이며 두 번째 주 조정 Codex를 실행하는 자리가 아닙니다.
5. 새 Codex 입력칸에 첫 요청문을 붙여 넣습니다. 명령·확인 터미널에서는 pwd로 실제 폴더를 확인합니다. 기획 세션은 P5가 끝날 때까지 유지합니다.
6. 기획 단계에는 일반 폴더도 가능하지만 별도 Git worktree를 만들 수는 없습니다. Git 연결은 G1에서 처리합니다. Orca에 폴더가 등록되지 않았다면 앱에서 먼저 열고 다시 시도합니다.

읽을 것

- 내 아이디어·기존 자료·프로젝트 AGENTS.md

남길 것

- docs/STATE.md
- docs/research/ 폴더

2. 표시된 입력 위치에 복사하세요

일반 터미널 · 현재 위치 확인

```
pwd
```

현재 작업공간에 기획 CLI 열기

```
orca-ide terminal create --worktree 'path:/home/gon/dev/apps/my-project' --title Codex --command 'codex --yolo' --focus --json
```

이미 열린 세션은 시작 옵션과 /status를 확인합니다. --yolo 적용이 확인되면 재사용하고, 아니면 현재 주 세션의 상태를 저장하고 종료한 뒤 이 명령으로 엽니다. 응답의 terminal handle을 STATE에 기록합니다. Orca에서 모델·effort를 따로 골랐다면 실행 전에 --command 안에 해당 CLI의 모델·effort 인수를 명시하세요.

Codex 입력칸 · 실행 상태 확인

```
/status
```

Codex 입력칸 · 기획 대화 이름

```
/rename 내 프로젝트-기획
```

주 조정 Codex에 보낼 요청문

```
현재 프로젝트는 내 프로젝트, 폴더는 /home/gon/dev/apps/my-project야. Orca 안의 Codex CLI로 P1~P5 기획을 진행하자.
현재 폴더와 기존 문서·AGENTS.md를 확인해. 기존 파일을 보존하고 docs/research/와 docs/STATE.md를 준비해.
이 세션은 기획 주 조정자야. P1~P5의 통합 문서는 네가 관리하고, 독립 조사는 Orca orchestration 작업자로 보낼 수 있어.
설치된 orca-cli와 orchestration 스킬의 버전별 가이드를 먼저 읽어. worker-start를 쓰는 경우 해당 도구의 대기·보고 규칙을 지켜.
현재 Codex의 /status에서 확인할 실행 위치·모델·권한·검색 상태를 안내해. 다른 환경의 설정이 그대로 적용된다고 가정하지 마.
STATE에는 현재 단계·다음 행동·미정만 먼저 기록해. 아직 구현·설치·원격 저장소 생성은 하지 말고 P1을 준비해.
새 주 모델과 협업 작업자의 실행 옵션은 Codex --yolo, Claude --dangerously-skip-permissions, Gemini --yolo로 전달해.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] Orca 작업공간과 실제 폴더가 일치한다.
- [ ] Codex가 실행된 기획 탭과 일반 명령 터미널을 구분했다.
- [ ] STATE.md를 실제로 열었다.

> **막히면**
>
> Orca 연결 오류는 실행 중인 앱과 선택된 작업공간부터 확인합니다. Linux 외부 터미널의 bare orca는 스크린리더일 수 있습니다. 이 안내의 직접 명령은 확인한 orca-ide를 사용합니다.

다음 · P1 · 같은 기획 세션

P1 아이디어를 질문과 답변으로 구체화한다

누구의 어떤 불편을 해결할지 한 문장으로 남깁니다.

열 창 기획 Codex 입력칸

세션 S0의 같은 기획 세션

---

1. 이 순서로 진행하세요

1. 아래 요청의 아이디어와 실제 불편을 내 말로 바꿉니다. AI의 질문에는 한 번에 1~3개씩 답합니다.
2. 모르면 “미정”, 추측이면 “가정”이라고 말합니다. 실제로 만나지 않은 사용자나 하지 않은 인터뷰를 기록하지 않습니다.

3. 한 덩어리의 문답이 끝나면 저장을 요청합니다. 파일 목록에서 docs/research/01-problem.md를 열고 내 답이 맞는지 읽습니다.
4. 예: “할 일 앱” → “내가 퇴근 후 할 일을 잊는다. 한 브라우저에서 간단히 기록하고 싶다.” 기능·기술은 아직 확정하지 않아도 됩니다.

읽을 것

- 내 답변·기존 자료

남길 것

- docs/research/01-problem.md
- docs/STATE.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
내 프로젝트 기획을 시작하자. 현재 프로젝트 폴더는 /home/gon/dev/apps/my-project야.
내 아이디어: [실제로 하고 싶은 일 한 문장으로 바꾸기]
실제로 불편했던 상황: [내 경험으로 바꾸기]
대상 사용자·현재 방법·불편·원하는 변화·시간/예산/환경 제약을 질문해줘.
한 번에 중요한 질문 1~3개만 하고 내 답을 기다려. 답하지 않은 내용을 지어내지 마.
문답이 한 덩어리 끝날 때마다 docs/research/01-problem.md를 갱신해줘.
질문/내 답변, 확인한 사실, 가정, 미정, 문제 한 문장, 다음 조사 질문을 구분해.
이 단계에서는 코드 구현과 도구 설치를 시작하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] 01-problem.md를 열어 내 실제 답변과 다른 내용이 없는지 확인했다.
- [ ] 문제 한 문장·대상·제약·미정이 구분되어 있다.

> **막히면**
>
> AI가 요약만 하고 파일을 만들지 않았다면 “지금 내용을 파일에 저장한 뒤 다시 읽어 경로와 핵심을 알려줘”라고 보냅니다.

다음 · P2 · 같은 기획 세션

P2 조사 질문과 작업자 담당 파일을 정한다

많이 검색하기보다, 어떤 답이 있어야 결정할 수 있는지 정합니다.

열 창 기획 Codex 입력칸

세션 주 조정자는 유지 · 조사 작업자는 독립 새 세션

1. 이 순서로 진행하세요

1. 문제 정의를 바탕으로 Q01, Q02처럼 질문을 나눕니다. 각 질문에 필요한 결정·출처·확인 방법·종료 기준을 붙입니다.
2. 예: Q01 기존 도구로 충분한가 / Q02 내가 실제로 겪는 빈도는 어느 정도인가 / Q03 저장 방식의 제약은 무엇인가.
3. AI가 인터넷으로 조사할 질문과 내가 직접 관찰·답변할 질문을 분리합니다. 사용자 경험은 웹 검색으로 대신 확인할 수 없습니다.
4. 서로 독립적인 질문만 나눕니다. 처음에는 조사 작업자 2개 이하로 시작합니다. 이는 이해를 돕기 위한 권장 규모이며 Orca의 기능 제한이 아닙니다.
5. 작업자 A는 notes/alternatives.md, B는 notes/constraints.md만 작성합니다. 주 조정자만 01~04 통합 문서와 STATE를 수정합니다. 작업자에게 내 채팅 전체가 자동 전달된다고 가정하지 않습니다.

읽을 것

- docs/research/01-problem.md

남길 것

- docs/research/02-research-plan.md
- docs/tasks/research-a.md
- docs/tasks/research-b.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
P1 파일을 읽고 docs/research/02-research-plan.md를 작성해줘.
Q번호 / 질문 / 필요한 결정 / 1차 출처 / 직접 확인 방법 / 종료 기준 / 담당 / 출력 파일을 표로 써.
웹에서 확인할 질문과 내가 실제로 답하거나 시험할 질문을 구분해.
독립적인 웹 조사 최대 2개를 제안하고 각각 docs/tasks/research-a.md, research-b.md에 자족적인 작업 지시서를 작성해.
각 지시서에는 목표, 읽을 파일, 담당 Q번호, 수정 허용 파일 1개, 금지 범위, 결과 형식, 완료 증거를 넣어.
작업자 A는 docs/research/notes/alternatives.md, B는 notes/constraints.md만 쓰게 해. 다른 작업자가 있으니 타인의 변경을 되돌리지 않도록 명시해.
01~04 통합 문서와 STATE는 주 조정자만 쓴다. 아직 작업자를 실행하지 말고 질문과 분담표부터 보여줘.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] 질문마다 완료 기준과 결과 파일이 있다.
- [ ] 서로 다른 작업자의 쓰기 파일이 겹치지 않는다.
- [ ] 사용자 확인이 필요한 질문은 별도로 남았다.

> **막히면**
>
> 질문 하나의 답을 다른 질문이 기다려야 한다면 병렬로 보내지 않습니다. 순서를 두거나 하나의 작업자가 수행하게 합니다.

P3

Orca 작업자를 실행하고 근거를 통합한다

주 조정자가 조사 작업을 맡기고 결과·질문·실패를 받아 처리합니다.

열 창 기획 Codex 입력칸

세션 기획 유지 · 작업자별 새 세션 · 실행 후 주 조정자 응답 대기

1. 이 순서로 진행하세요

1. 아래 요청을 보내면 주 조정자가 worker-start로 독립 작업을 시작합니다. 작업자를 직접 여럿 열고 같은 기획 세션을 resume하지 않습니다.
2. 주 조정자는 독립 작업을 시작한 뒤 현재 응답을 끝내고 Orca의 알림으로 다시 동작합니다. 이 “응답 종료”는 Codex 종료 명령이나 탭 닫기가 아닙니다. 기획 탭을 그대로 둡니다.
3. 작업자가 물으면 주 조정자가 이미 정해진 범위에서 답합니다. 사용자 선택이 필요한 문제만 내게 묻습니다. 지시를 바꾸고 싶으면 작업자 탭에 직접 타이핑하기보다 주 조정자에게 전달합니다.
4. 작업 완료 보고(worker_done)가 오면 주 조정자가 실제 결과 파일·출처·누락을 읽습니다. 시작 성공과 완료 보고만으로 근거가 검증된 것은 아닙니다.
5. 작업자의 메모를 03-evidence.md의 S번호 출처표로 합칩니다. 내 관찰은 U번호, AI 해석은 추론으로 구분합니다. 중요한 링크는 원문을 열어 확인합니다.

읽을 것

- 02-research-plan.md
- docs/tasks/research-*.md
- docs/research/notes/*.md

남길 것

- docs/research/notes/alternatives.md · 작업자 A
- docs/research/notes/constraints.md · 작업자 B
- docs/research/03-evidence.md · 주 조정자
- docs/STATE.md · Run/Task/Dispatch 기록

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
확정된 조사 분담표대로 Orca orchestration 작업자를 실행해줘.
일반 폴더 또는 현재 작업공간을 공유하되 각자 지정한 결과 파일만 쓰게 해. 작업자마다 CLI 승인 생략 옵션을 포함해 실행해. Codex는 codex --yolo, Claude·Opus·Fable은 claude --dangerously-skip-permissions, Gemini는 gemini --yolo를 사용해.
현재 Orca worker-start에는 CLI 인수 전달 옵션이 없으므로 --agent로 곧바로 시작하는 대신, 해당 작업공간에서 terminal create --command로 CLI를 열어.
반환된 terminal handle을 terminal wait --for tui-idle로 확인한 뒤 worker-start --spec-file <지시서 절대경로> --terminal <handle> --worktree <그 작업공간 selector> --json으로 연결해.
내가 모델을 지정했다면 CLI 실행 명령에 그 모델·effort 옵션을 전달해. 미지정이면 기존 Orca 선택이나 현재 주 조정자의 모델·effort를 확인해 CLI 명령에 명시하고 STATE에 기록해. CLI 기본값이 Orca 설정과 같다고 가정하지 마.
terminal wait timeout이나 worker-start 실패가 나면 기존 handle과 오류를 기록하고 상태를 먼저 확인해. 같은 작업의 터미널을 반복 생성하지 마. 재시도에도 동일 Task와 원래 배치를 유지하고 승인 생략 옵션으로 실행한 터미널을 --terminal로 연결해. 복구 가이드의 liveness·next·release 조건을 따라.
각 작업의 Run·Task·Dispatch ID, 담당 파일, 상태를 STATE에 기록해. 독립 작업을 시작한 뒤 설치 가이드대로 응답을 끝내 Orca 알림을 기다려. 반복 check 폴링 스크립트는 만들지 마.
질문은 ask/reply, 추가 지시는 Dispatch 대상 send로 처리해. 실패한 작업을 성공으로 표시하지 마.
모든 작업이 정리되면 원문과 결과 파일을 확인하고 03-evidence.md로 통합해.
출처마다 S번호·기관/제목·URL·게시/수정일·확인일·관련 Q번호·확인 사실·한계를 적어. 날짜를 모르면 모름으로 써.
직접 관찰은 U번호로, 해석과 미확인은 따로 적어. 핵심 근거가 부족하면 P2의 추가 조사로 돌아가.

3. 실제로 확인한 항목만 체크하세요

- [ ] Run·Task·Dispatch와 실제 결과 파일을 확인했다.
- [ ] 모든 주요 결론에 S번호 또는 U번호가 연결된다.
- [ ] 작업 실패·접근 불가·충돌 근거가 숨겨지지 않았다.

막히면

작업자 시작이 실패하면 같은 명령을 무작정 재실행하지 않습니다. failedStage와 잔여 작업 상태를 확인하고 설치 recovery 가이드를 따릅니다. 무응답·시간 초과만으로 종료됐다고 판단하지 않습니다.

다음 · P4 · 같은 기획 세션

P4 대안을 비교하고 실제 선택을 기록한다

조사 결과에서 개발할 범위와 하지 않을 일을 고릅니다.

열 창 기획 Codex 입력칸

세션 같은 기획 세션

1. 이 순서로 진행하세요

1. 기존 도구 사용, 최소 범위 직접 개발, 추가 조사·보류를 같은 기준으로 비교합니다. 기술 도입부터 결정하지 않습니다.
2. 내 선택을 바꿀 큰 가정 1~2개만 직접 확인합니다. 실제로 확인한 날짜·행동·결과를 전달합니다. 못 했다면 못 했다고 기록합니다.
3. 직접 개발을 선택했다면 D01, D02에 결정·이유·S/U 근거·제외 범위·재검토 조건을 적습니다.
4. 학습용으로 개발하는 선택도 가능합니다. 그 경우 “시장 수요를 입증했다”는 표현을 쓰지 않습니다. 개발하지 않기로 했다면 결정 파일과 STATE를 남기고 여기서 마칩니다.

읽을 것

- 01-problem.md
- 02-research-plan.md
- 03-evidence.md

남길 것

- docs/research/04-decision.md
- docs/STATE.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
docs/research/01-problem.md, 02-research-plan.md, 03-evidence.md를 읽고 선택지를 비교해줘.
기존 도구 사용 / 최소 범위 직접 개발 / 보류를 내 목표·시간·비용·운영 부담으로 비교해.
중요한 주장은 S번호 또는 U번호와 연결하고 부족한 근거를 표시해.
결정을 바꿀 가정 1~2개, 직접 검증 방법, 통과 기준을 제안한 뒤 내 실제 확인 결과를 기다려.
직접 확인하지 않은 인터뷰·사용·수요·실험을 지어내지 마.
내 선택을 받으면 docs/research/04-decision.md에 D번호·선택·이유·근거·제외 범위·미확인·재검토 조건을 저장해.
학습 목적의 실습이라면 시장 수요 검증과 구분해. 아직 최종 기획 승인이나 구현 완료로 표시하지 마.
문서에는 확정 사실·사용자 결정·AI 제안·미확인을 구분해줘. 파일을 실제로 저장하고 다시 읽어 경로와 핵심 내용을 알려줘. 저장할 수 없으면 저장했다고 말하지 말고 이유와 본문을 보여줘. 기존 문서는 읽고 필요한 부분만 갱신해줘.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] 04-decision.md의 결정이 내 선택과 일치한다.
- [ ] 실제 검증·가정·미확인과 재검토 조건이 구분되어 있다.

> **막히면**
>
> 중요한 미정이 있으면 승인했다고 적지 않습니다. 근거가 부족하면 P3, 문제 자체가 바뀌면 P1로 돌아갑니다.

다음 · 직접 개발 → P5 / 추가 확인 → P3 / 보류 → 기록 후 종료

P5

최종 개발 문서 3종을 만든다

조사 문서를 근거로 남기고, 개발자가 바로 읽을 실행 기준을 만듭니다.

열 창 기획 Codex 입력칸 → 파일 목록에서 최종 문서 열기

세션 같은 기획 세션의 마지막 단계

1. 이 순서로 진행하세요

1. P1~P4를 지우거나 한 파일에 원문 그대로 붙이지 않습니다. 조사 기록은 근거로 보존하고 필요한 결론만 최종 문서로 옮깁니다.
2. PROJECT-BRIEF.md에는 목표·사용자·첫 출시 포함/제외·사용 흐름·데이터·요구 R번호·완료 기준 AC번호·결정 D번호·미정·버전·확인 상태를 넣습니다.
3. TASKS.md에는 첫 구현부터의 작업 순서, 선행 작업, 읽을 문서, 담당 파일 범위, 시험할 AC번호를 씁니다. STATE.md에는 지금 위치와 다음 행동, Run/Task/Dispatch, 기준 커밋, GitHub 링크를 둡니다.
4. 예: U02 “새로고침 뒤에도 남겨야 함” → D02 저장 필요 → R04 새로고침 유지 → AC04 항목과 완료 상태 일치 → TASK 저장 구현. 연결이 끊기면 자료 또는 결정을 다시 확인합니다.
5. 내가 문서를 읽고 범위를 확인한 뒤에만 실제 확인 날짜와 버전을 기록합니다. AI가 확인을 지어내면 안 됩니다. 미정은 담당·해결 시점·막는 작업을 적습니다.
6. 새 개발 AI가 3개 문서만 읽어도 “무엇을, 어디까지, 어떻게 확인할지, 먼저 무엇을 할지” 설명할 수 있으면 G1로 넘어갑니다.

읽을 것

- P1~P4 통합 문서·원문 근거·실제 사용자 선택

남길 것

- docs/PROJECT-BRIEF.md
- docs/TASKS.md
- docs/STATE.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
P1~P4 파일을 실제로 읽고 개발 시작용 문서 3종을 만들어줘. 조사 원본은 보존해.
1) docs/PROJECT-BRIEF.md: 목표·사용자·문제·첫 출시 포함/제외·사용 흐름·데이터·기술 결정과 미정·R번호 요구·관찰 가능한 AC번호·관련 D번호·근거 링크·가정·재검토 조건·버전·내 확인 상태.
2) docs/TASKS.md: 첫 작업부터 순서·선행 관계·담당 파일·각 작업이 구현할 R/검증할 AC·결과 증거. 아직 코드 구조가 없으면 파일 범위는 후보로 표시하고 개발 준비에서 확정해.
3) docs/STATE.md: 현재 단계·읽을 파일·미정·다음 행동·작업자 Run/Task/Dispatch와 상태·Git 브랜치/커밋·GitHub URL. 없는 값은 아직 없음으로 써.
중요한 요구는 Q → S/U → D → R/AC → 작업 연결표로 확인해. 새 근거 없이 기능을 추가하지 마.
내가 문서를 읽고 확인하기 전에는 개발 시작 승인으로 쓰지 마. 시작을 막는 미정과 구현하면서 풀 수 있는 미정을 나누고, 승인할 정확한 버전과 범위를 보여줘.
확인 후에는 실제 확인 내용을 반영해 저장하고 세 파일을 다시 읽어 누락·모순·끊어진 링크를 확인해. 마지막에 새 개발 세션에 넘길 한 단락 인계문을 써줘.

3. 실제로 확인한 항목만 체크하세요

- [ ] 최종 문서 3종을 실제로 열어 내 결정과 비교했다.
- [ ] 각 R에 확인할 수 있는 AC가 있고 D와 근거로 거슬러 갈 수 있다.
- [ ] 첫 작업·제외 범위·미정 처리·사용자 확인 상태가 분명하다.

> **막히면**
>
> “멋진 앱을 만든다”는 개발 완료 기준이 아닙니다. “공백만 넣으면 저장하지 않고 안내한다”처럼 직접 시험할 수 있게 고칩니다. 큰 미정은 숨기지 않고 해당 작업을 막아 둡니다.

다음 · G1 · 기획 세션에서 GitHub에 기준 문서 저장

G1

GitHub에 기획 기준을 저장한다

작업자들이 같은 문서 버전에서 출발하고 원격에도 기록이 남게 합니다.

열 창 기획 Codex 입력칸 + 로그인 필요 시 사용자가 여는 인증 화면

세션 같은 기획 세션 · 원격 작업은 아래 요청에서 범위 지정

---

1. 이 순서로 진행하세요

1. 신규 프로젝트인지 기존 저장소인지 선택합니다. 기존 저장소에서는 원격 주소·기본 브랜치·현재 변경·보호 규칙을 먼저 확인합니다. 새 저장소로 바꾸지 않습니다.

2. 일반 터미널에서 gh auth status로 로그인 상태를 봅니다. 로그인이 필요하면 사용자가 브라우저 인증을 완료합니다. 토큰·인증코드를 문서나 AI 입력칸에 붙이지 않습니다.
3. 처음 연결하는 프로젝트는 아래 “신규 원격 연결” 요청을, 이미 GitHub 원격이 있는 프로젝트는 “기존 저장소” 요청을 사용합니다. 이미 Orca가 만든 로컬 Git은 보존합니다. 두 요청을 모두 실행하지 않습니다.
4. 업로드할 문서에 비밀값·불필요한 개인정보가 없는지 확인합니다. 검토한 파일만 커밋하고, 실제 GitHub에서 파일 내용과 커밋을 다시 읽습니다.
5. 기존 저장소에서 기획 PR이 아직 기본 브랜치에 반영되지 않았다면 개발 worktree를 기본 브랜치에서 만들면 문서가 빠질 수 있습니다. 기획을 먼저 정상 반영하거나 명시한 기획 커밋을 작업 기준으로 지정합니다.
6. Orca에 저장소가 등록됐는지 repo list로 확인합니다. 등록되지 않았다면 repo add로 연결합니다. 이는 로컬 폴더를 Orca에 등록하는 일이며 GitHub 업로드와는 다릅니다.

읽을 것

- 최종 문서 3종·P1~P4
- 현재 Git 상태와 원격 저장소

남길 것

- 기획 기준 커밋
- GitHub 저장소 URL·기획 PR URL(기존)
- docs/STATE.md의 원격 기록

2. 표시된 입력 위치에 복사하세요

일반 터미널 · GitHub 인증 상태만 확인

```
gh auth status
```

로그인이 필요할 때만 · 사용자가 일반 터미널에서 실행

```
gh auth login --web --git-protocol https
```

브라우저 인증 안내를 직접 완료합니다. 토큰이나 일회용 코드를 AI 채팅·문서에 붙이지 않습니다.

일반 터미널 · Orca 저장소 등록 여부 확인

```
orca-ide repo list --json
```

목록에 없을 때만 · 기존 로컬 저장소 등록

```
orca-ide repo add --path '/home/gon/dev/apps/my-project' --json
```

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
/home/gon/dev/apps/my-project의 Git 상태·기존 원격·현재 브랜치·미커밋 변경·GitHub 인증 여부를 확인해줘. 비밀값은 출력하지 마.
이미 로컬 Git이 있어도 GitHub 원격이 없다면 신규 원격 연결 경로로, 원격이 있으면 기존 저장소 경로로 안내해. 잘못된 원격을 임의로 바꾸지 마.
기획 문서 중 원격에 저장할 파일 목록과 현재 기준 커밋을 보여줘. 아래에서 내 상황에 맞는 요청을 이어서 보낼게.

원격이 없을 때만 · 신규 원격 연결 요청

/home/gon/dev/apps/my-project의 프로젝트를 [GitHub 소유자/저장소 입력 필요]라는 새 비공개 GitHub 저장소에 연결해줘.
이미 있는 로컬 Git은 그대로 사용해. Git도 없다면 내가 지정한 이 프로젝트만 Git으로 초기화해. 기존 원격이 발견되면 덮어쓰지 말고 알려줘.
검토한 기획 문서를 커밋하고 새 비공개 원격에 push해줘.
공통: 사용자 인증은 안전한 브라우저 흐름으로 안내하고 비밀값을 출력하지 마. 검토한 문서에 비밀값·무관한 파일이 없는지 확인하고 명시한 파일만 커밋해.
실제 GitHub에서 저장소·브랜치·문서 내용·커밋을 다시 읽어 확인해. 기본 브랜치와 다음 작업 기준 커밋을 STATE에 기록해.
Orca에 이 로컬 저장소가 없다면 repo add로 등록하고 repo ID를 확인해. 등록, push, PR 생성, 병합을 구분해 보고해.

원격이 이미 있을 때만 · 기존 저장소 요청

/home/gon/dev/apps/my-project의 기존 저장소와 원격을 유지해. 연결 대상 [GitHub 소유자/저장소 입력 필요]가 실제 원격과 맞는지 확인해줘.
기획 변경을 별도 브랜치에 커밋·push하고 기획 PR을 만들어줘. 기존 정책의 필수 검사·리뷰가 충족되면 정상 병합까지 진행해. 강제 push나 보호 규칙 우회는 하지 마.
공통: 사용자 인증은 안전한 브라우저 흐름으로 안내하고 비밀값을 출력하지 마. 검토한 문서에 비밀값·무관한 파일이 없는지 확인하고 명시한 파일만 커밋해.
실제 GitHub에서 저장소·브랜치·문서 내용·커밋을 다시 읽어 확인해. 기본 브랜치와 다음 작업 기준 커밋을 STATE에 기록해.
Orca에 이 로컬 저장소가 없다면 repo add로 등록하고 repo ID를 확인해. 등록, push, PR 생성, 병합을 구분해 보고해.

3. 실제로 확인한 항목만 체크하세요

- [ ] 실제 GitHub URL에서 최종 문서와 내용이 보인다.
- [ ] 기본 브랜치와 개발 기준 커밋을 알고 있다.
- [ ] Orca가 이 저장소를 인식한다.

> **막히면**
>
> 인증·조직 권한·필수 리뷰로 막히면 구체적인 미충족 조건을 남깁니다. 저장소 생성 수락이나 push 성공 메시지만으로 문서가 맞게 올라갔다고 단정하지 않습니다.

다음 · D1 · 기획 기록을 닫고 새 개발 조정 세션

새 개발 조정 세션에 문서로 인계한다

D1

긴 조사 대화 대신 확정 문서와 작업 상태를 기준으로 개발합니다.

열 창 기획 Codex에서 기록 완료 → 새 개발 Codex CLI

세션 기획 종료 후 새 개발 세션 · 기존 기획 resume 아님

1. 이 순서로 진행하세요

1. 기획 조정자에게 모든 작업자의 완료·차단 여부와 다음 행동을 STATE에 저장하게 합니다. 작업자가 아직 실행 중이면 기존 조정자를 유지하고 먼저 정리합니다.
2. 기획 Codex에서 /quit를 입력합니다. 이 종료는 작업자를 자동으로 멈추거나 Git 상태를 되돌리는 기능이 아닙니다.
3. G1에서 확인한 저장소에 새 개발 Codex CLI를 엽니다. /rename으로 프로젝트-개발 이름을 붙입니다. 기획 대화를 resume하지 않습니다.
4. 아래 인계 요청을 보내고 AI가 파일을 실제로 읽어 요약하는지 확인합니다. 다른 창의 대화를 안다고 주장하면 파일 경로를 다시 지정합니다.
5. 기술·폴더 구조·테스트 명령이 미정이면 기존 프로젝트 구성을 먼저 조사해 확정합니다. 할 일 앱 예시의 React 선택을 내 프로젝트에 자동 적용하지 않습니다.
6. Gon OS의 gon-project-lifecycle을 현재 스킬 목록에서 확인합니다. 웹 작업이면 gon-web-engineering도 해당 범위에 적용합니다. 개발용 제품·구조 문서가 이미 있으면 그 문서에 연결하고, 새로 필요한 경우에만 PRODUCT.md·ARCHITECTURE.md를 준비합니다.

읽을 것

- docs/PROJECT-BRIEF.md
- docs/TASKS.md
- docs/STATE.md
- 프로젝트 AGENTS.md·실제 빌드 설정

남길 것

- 개발 준비 결과·확정한 작업 파일 범위
- 갱신된 docs/TASKS.md와 docs/STATE.md
- 필요한 경우 PRODUCT.md·ARCHITECTURE.md 또는 기존 동등 문서

2. 표시된 입력 위치에 복사하세요

기획 Codex 입력칸 · 기록 후 종료

```
/quit
```

현재 작업공간에 개발 CLI 열기

```
orca-ide terminal create --worktree 'path:/home/gon/dev/apps/my-project' --title Codex --command 'codex --yolo' --focus --json
```

이미 열린 세션은 시작 옵션과 /status를 확인합니다. --yolo 적용이 확인되면 재사용하고, 아니면 현재 주 세션의 상태를 저장하고 종료한 뒤 이 명령으로 엽니다. 응답의 terminal handle을 STATE에 기록합니다. Orca에서 모델·effort를 따로 골랐다면 실행 전에 --command 안에 해당 CLI의 모델·effort 인수를 명시하세요.

새 개발 Codex 입력칸 · 이름 지정

```
/rename 내 프로젝트-개발
```

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
$gon-project-lifecycle
이 세션은 내 프로젝트의 개발 주 조정자야. /home/gon/dev/apps/my-project에서 AGENTS.md와 docs/PROJECT-BRIEF.md, TASKS.md, STATE.md를 실제로 읽어줘.
목표·포함/제외·R/AC·미정·현재 GitHub/브랜치/기준 커밋·첫 작업을 요약해. 조사 채팅을 알고 있다고 가정하지 마.
실제 승인 상태와 기획 기준 커밋을 확인하고, 기술 미정은 기존 구성과 제약을 보고 설명해. 시작을 막는 결정만 내게 물어봐.
기존 패키지 관리자를 유지하고 실제 빌드·검사 명령을 확인해. 준비가 끝나면 TASKS의 파일 담당과 선행 관계를 확정해.
다음 단계 D2에서 Orca 작업자를 분담할 수 있도록 작은 첫 구현 범위를 제안해. 아직 작업자 실행이나 전체 기능 구현은 시작하지 마.
현재 로더에서 스킬을 찾지 못하면 있다고 가정하지 말고 알려줘. 웹 프로젝트라면 사용 가능한 gon-web-engineering의 관련 절만 적용하고 기존 스택은 유지해.
새 주 모델과 협업 작업자의 실행 옵션은 Codex --yolo, Claude --dangerously-skip-permissions, Gemini --yolo로 전달해.
```

인계와 기술 선택을 확인한 뒤 · 개발 기준·실행 환경 준비 요청

```
$gon-project-lifecycle
앞서 확인한 내 프로젝트의 문서·기술 결정을 기준으로 첫 구현에 필요한 문서와 실행 환경을 준비해줘.
웹 프로젝트라면 현재 사용 가능한 gon-web-engineering의 관련 부분만 적용해.
기존 제품·구조 문서가 있으면 그것을 갱신해. 없고 필요한 새 프로젝트라면 docs/PRODUCT.md에 확정된 목표·R/AC를, docs/ARCHITECTURE.md에 기술·데이터·저장·구성·제약을 기록해. PROJECT-BRIEF의 버전과 결정 링크를 붙이고 별도 기획을 만들어내지 마.
AEOS의 PROJECT.md·ARCHITECT.md가 있으면 그 경로를 유지하고 같은 내용을 중복 생성하지 마.
GitHub의 Issues/Projects를 실행 상태의 기준으로 삼고 TASKS·STATE에는 실제 URL을 가진 요약과 Orca 작업 대응만 남겨. 기존 관리 정책·필드·라벨을 보존해.
실제 패키지 관리자와 잠금 파일을 유지해 첫 개발 환경을 준비하고, 실행할 명령·실제 로컬 URL·검사 명령을 알려줘. 스택이 미정이면 결정된 범위까지만 진행해.
준비한 파일과 기준 커밋을 확인하고, 전체 기능 구현은 D2/D3의 작업 분담에 따라 진행하자.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] 새 개발 세션이 실제 파일에서 범위와 첫 작업을 정확히 설명했다.
- [ ] 미정·기술·검사 명령·기준 커밋이 확인됐다.
- [ ] 활성 기획 작업자가 남아 있지 않거나 기존 조정자가 명확히 맡고 있다.

> **막히면**
>
> 새 대화는 문서가 없으면 이전 합의를 모릅니다. 반대로 Git 브랜치·worktree를 새로 만들었다고 AI 대화가 자동으로 이어지는 것도 아닙니다.

다음 · D2 · 같은 개발 조정 세션

D2 GitHub 이슈와 Orca 작업을 연결한다

무엇을 왜 만드는지는 이슈, 지금 누가 하는지는 Orca에 남깁니다.

열 창 개발 Codex 입력칸

세션 같은 개발 조정 세션

1. 이 순서로 진행하세요

1. 첫 출시 전체를 무조건 동시에 구현하지 않습니다. 공통 인터페이스·데이터 형식·기초 화면이 먼저 필요하면 선행 작업으로 둡니다.
2. 독립 작업의 파일 범위가 겹치면 별도 worktree를 만듭니다. 같은 폴더에서 조사 파일만 따로 쓰는 방식과 코드 병렬 작업의 충돌 위험은 다릅니다.
3. 같은 목적의 기존 GitHub 이슈를 먼저 찾습니다. 없으면 목표·R/AC·포함/제외·검증 방법을 담은 이슈를 만듭니다. 작은 작업은 이슈 하나에 체크리스트로 묶을 수 있습니다.
4. Orca Run은 이번 개발 묶음, Task는 작업 항목, Dispatch는 실제 실행 한 번입니다. GitHub 이슈 번호와 이 ID들은 서로 다른 값입니다.
5. TASKS.md에 작업→이슈 URL→선행 작업→담당 파일→완료 증거를 남기고, STATE에 Run ID와 기준 브랜치/커밋을 기록합니다. 작업자가 모든 문서를 공동 편집하지 않게 합니다.
6. GitHub에 연결된 뒤 실행 상태의 기준은 Issues/기존 Projects입니다. TASKS와 STATE는 URL을 가진 로컬 요약·Orca 대응표로 유지합니다. 기존 다른 추적기가 지정돼 있으면 임의로 이전하지 않습니다.

읽을 것

- 최종 문서 3종·확정 코드 구조·기준 커밋

남길 것

- GitHub 이슈
- docs/TASKS.md
- docs/tasks/dev-*.md
- docs/STATE.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
첫 출시 범위를 작은 작업으로 나누고 GitHub의 기존 이슈를 확인해. 같은 목적의 이슈가 없으면 [GitHub 소유자/저장소 입력 필요]에 만들어 줘.
각 작업에는 목표 R/AC, 포함/제외, 선행 작업, 읽을 파일, 독점 수정 파일, 실제 검증 명령·동작, 결과 보고 형식을 적어.
공통 기반이 필요한 작업은 선행 Task로 두고, 선행 작업이 통합되기 전에는 후속 작업자를 시작하지 마.
작업 지시는 docs/tasks/dev-*.md에 저장해. 작업자는 타인의 변경을 되돌리지 않으며 자기 파일만 변경하고, 자기 결과를 검증·커밋·push한 뒤 draft PR로 보고하게 해.
주 조정자가 TASKS/STATE 통합 기록과 최종 통합을 담당해. 각각의 이슈 URL과 Orca Task 대응을 적어.
갱신한 기준 문서·TASKS·작업 지시를 검토한 브랜치에 커밋하고 그 기준 ref와 커밋을 기록해. 기존에 승인된 원격 저장 범위에서 push하고, 모든 작업자가 이 커밋부터 시작하도록 해. 이 단계에서는 분담표와 생성 이슈를 보여줘.
기존 GitHub Project가 있으면 현재 필드·상태를 그대로 사용해 연결해. 새 Project를 매번 만들지 말고, TASKS는 GitHub URL을 가진 작업자 분담 요약으로 유지해. 원격 동기화가 안 되면 pending_sync로 남겨.

3. 실제로 확인한 항목만 체크하세요

- [ ] 병렬 작업의 파일 책임과 선행 관계가 명확하다.
- [ ] 이슈 URL과 Task가 연결된다.
- [ ] 작업 지시와 기준 문서가 모든 작업자의 출발점에 있다.

> **막히면**
>
> 공통 파일을 여러 작업자가 고치게 되면 분담을 줄이거나 공통 작업을 먼저 완료합니다. 작업자 수를 늘리는 것이 목표는 아닙니다.

다음 · D3 · 독립 작업부터 실행

D3 개발 작업자를 분리해 실행한다

독립 작업은 병렬로, 의존하는 작업은 통합 후 순서대로 진행합니다.

열 창 개발 Codex 입력칸

세션 주 조정자 유지 · 작업자별 새 세션과 별도 worktree

---

1. 이 순서로 진행하세요

1. 공통 기반 작업이 있으면 먼저 실행·검증·통합합니다. 후속 작업자의 새 worktree가 그 통합 커밋에서 시작하는지 확인합니다.
2. 서로 독립적인 코드 작업은 Orca worktree create로 작업공간만 분리한 뒤, 승인 생략 옵션으로 연 각 CLI 터미널을 worker-start --terminal로 연결합니다. repo ID와 기준 브랜치는 실제 값으로 지정합니다.
3. 주 조정자가 시작한 작업자의 Dispatch·worktree·브랜치를 기록합니다. 시작 응답의 ready는 일을 끝냈다는 뜻이 아닙니다.

4. 주 조정자가 응답을 끝내면 그대로 기다립니다. 다른 탭을 봐도 되지만 주 조정 세션을 /new로 바꾸거나 닫지 않습니다. 추가 요구는 주 조정자에게 보냅니다.
5. 작업자가 끝나면 변경 파일·검사 결과·커밋·draft PR·남은 문제를 함께 받습니다. 새 worktree 작업자는 보존될 수 있으며, 조정자가 실제 상태와 정리 가능한 조건을 확인합니다.

읽을 것

- docs/tasks/dev-*.md
- 선행 작업이 포함된 기준 커밋

남길 것

- 작업자별 코드·검증·커밋·draft PR
- 주 조정자 STATE 업데이트

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
확정한 작업 분담을 Orca orchestration으로 실행해줘.
Git 저장소와 기준 브랜치/커밋을 확인하고, 충돌 가능한 코드 작업은 서로 다른 새 worktree에 배치해. Orca worktree create로 repo·기준 브랜치를 지정해 작업공간만 먼저 만들고, 그 경로에서 승인 생략 옵션을 넣어 CLI 터미널을 열어.
선행 작업은 먼저 검증·통합하고, 후속 작업자는 그 통합 커밋을 기준으로 시작해. 서로 독립적인 준비된 작업만 병렬 실행해.
작업자마다 CLI 승인 생략 옵션을 포함해 실행해. Codex는 codex --yolo, Claude·Opus·Fable은 claude --dangerously-skip-permissions, Gemini는 gemini --yolo를 사용해.
현재 Orca worker-start에는 CLI 인수 전달 옵션이 없으므로 --agent로 곧바로 시작하는 대신, 해당 작업공간에서 terminal create --command로 CLI를 열어.
반환된 terminal handle을 terminal wait --for tui-idle로 확인한 뒤 worker-start --spec-file <지시서 절대경로> --terminal <handle> --worktree <그 작업공간 selector> --json으로 연결해.
내가 모델을 지정했다면 CLI 실행 명령에 그 모델·effort 옵션을 전달해. 미지정이면 기존 Orca 선택이나 현재 주 조정자의 모델·effort를 확인해 CLI 명령에 명시하고 STATE에 기록해. CLI 기본값이 Orca 설정과 같다고 가정하지 마.
terminal wait timeout이나 worker-start 실패가 나면 기존 handle과 오류를 기록하고 상태를 먼저 확인해. 같은 작업의 터미널을 반복 생성하지 마. 재시도에도 동일 Task와 원래 배치를 유지하고 승인 생략 옵션으로 실행한 터미널을 --terminal로 연결해. 복구 가이드의 liveness·next·release 조건을 따라.
각 작업자의 Dispatch·브랜치·worktree·이슈를 STATE에 기록하고, 작업자는 자신의 검증과 커밋·push·draft PR URL을 worker_done으로 보고하게 해.
시작 후에는 설치 가이드대로 응답을 끝내 Orca 알림을 기다려. 추가 지시는 send, 질문은 ask/reply로 처리해. 실패한 작업을 완료로 바꾸지 마.
반환된 next가 있으면 따르고, 결과를 받으면 작업별 성공·실패·차단과 실제 증거를 정리해.

3. 실제로 확인한 항목만 체크하세요

- [ ] 작업자마다 실제 작업공간·브랜치·담당 파일이 다르다.
- [ ] 모든 작업자가 올바른 기준 문서를 읽었다.
- [ ] 완료·실패·미응답을 분리해 확인했다.

막히면

실패 재시도는 같은 Task에 실패 Dispatch를 연결합니다. 시간 초과만으로 새 작업을 중복 생성하지 않습니다. 정확한 재시작은 설치 recovery 가이드와 현재 liveness/next를 따릅니다.

다음 · D4 · 작업자 결과 검수·통합

D4 결과를 검수하고 통합 상태를 시험한다

작업자별 성공을 하나의 제품 동작으로 확인합니다.

열 창 개발 Codex 입력칸 + 실제 실행 화면

세션 같은 개발 조정 세션 · 수정은 해당 작업자 후속 작업

---

1. 이 순서로 진행하세요

1. 작업자가 보고한 코드 변경·검사·커밋·PR이 실제로 존재하는지 확인합니다. 로그에 PASS만 적힌 것과 내가 원하는 AC 동작은 다릅니다.
2. 수정이 필요하면 주 조정자가 기존 Dispatch에 send로 전달합니다. 로컬의 완료·닫힌 작업자는 후속 Task로 이어질 수 있습니다. 새로 비슷한 작업자를 계속 만들지 않습니다.
3. 각 PR의 필수 검사·리뷰와 승인된 병합 범위를 확인합니다. 검토가 끝난 draft PR은 준비 상태로 전환한 뒤, 준비된 순서대로 정상 병합하거나, 아직 병합할 수 없다면 별도 통합 브랜치에서 조합을 시험하고 상태를 남깁니다.
4. 통합한 상태에서 실제 프로젝트의 빌드·테스트를 실행하고 R/AC에 따라 화면을 직접 조작합니다. 작업자별 검사 통과는 합쳐진 결과의 성공을 보장하지 않습니다.
5. 예: 빈 입력, 정상 입력, 새로고침, 저장 실패처럼 이번에 구현한 범위의 성공·실패 상태를 확인합니다. 아직 구현하지 않은 기능을 통과했다고 표시하지 않습니다.

읽을 것

- 작업자 변경·보고·PR
- PROJECT-BRIEF의 AC
- 필수 검사·프로젝트 리뷰 규칙

남길 것

- 통합 검증 기록
- 수정 후속 Task
- docs/STATE.md

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
작업자별 결과 파일·diff·검사·커밋·PR을 확인해. 빠진 증거와 모순이 있을 때 해당 범위만 추가 확인해.
수정은 해당 Dispatch에 send로 돌려보내고 후속 Task 결과를 받아줘.
준비된 PR은 기존 권한·필수 검사·리뷰 조건 안에서 정상 병합까지 진행해. 보호 규칙 우회·강제 push는 하지 마.
전체 변경이 합쳐진 커밋에서 프로젝트의 실제 빌드·검사를 수행하고, 이번에 구현한 AC를 직접 실행해 확인해.
AC별 실제 동작·증거·실패·미검증을 기록해. 화면 확인이 필요하면 어떤 화면에서 무엇을 눌렀는지 남겨.
작업자 종료나 CI 성공을 제품 전체 완료와 혼동하지 말고, 남은 문제가 있으면 같은 개발 범위 안에서 수정·검증을 이어가.

3. 실제로 확인한 항목만 체크하세요

- [ ] 실제 통합 커밋에서 검사했다.
- [ ] 이번 범위의 AC 동작을 실제로 확인했다.
- [ ] 실패와 미검증이 기록에 남는다.

막히면

서로 다른 worktree의 파일은 자동으로 합쳐지지 않습니다. PR 병합 또는 검토한 통합이 필요합니다. 충돌이 있으면 파일 책임과 최신 기준을 확인하고 관련 변경만 조정합니다.

다음 · G2 · GitHub 원격 상태 최종 확인

G2 PR·CI·이슈의 최종 상태를 확인한다

로컬 저장, 원격 push, PR 병합, 자동 검사를 각각 확인합니다.

열 창 개발 Codex 입력칸 + GitHub 저장소의 PR/Actions/Issues 화면

세션 같은 개발 조정 세션

---

1. 이 순서로 진행하세요

1. PR에서 변경 목적·포함 범위·검증·남은 문제를 읽습니다. 해당 PR 하나가 이슈 전체를 해결할 때만 Closes #번호를 사용합니다.
2. 기본 브랜치 대상 PR의 종료 키워드는 병합 시 이슈를 닫을 수 있습니다. 여러 PR이 나누어 해결하는 이슈라면 마지막 완료 PR에만 종료 키워드를 두고, 중간 PR은 Refs #번호 등으로 참조합니다.
3. GitHub Checks/Actions에서 최신 커밋의 필수 검사를 확인합니다. 검사 자체가 없거나 건너뛰었다면 “CI 통과”라고 하지 않습니다.
4. 원격 PR의 실제 merged 상태, 기본 브랜치의 커밋과 파일, 이슈의 최종 상태를 확인합니다. 작업자 종료와 PR 생성만으로 병합됐다고 말하지 않습니다.

5. Orca 작업공간에도 이슈/PR URL을 연결하면 나중에 관련 작업을 찾기 쉽습니다. 이 연결은 GitHub 이슈를 닫거나 PR을 병합하는 기능이 아닙니다.

읽을 것

- PR·이슈·CI 실행·기본 브랜치

남길 것

- 최종 GitHub URL과 커밋
- docs/STATE.md의 전달 상태

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
[GitHub 소유자/저장소 입력 필요]의 이번 작업을 최종 확인해줘.
작업별 PR URL·최종 head 커밋·기본 브랜치 반영 커밋·merged 여부·최신 필수 CI 결과·이슈 상태를 실제로 조회해.
CI 없음·pending·skipped·실패를 통과와 구분해. 중간 PR이 전체 이슈를 너무 일찍 닫지 않았는지 확인해.
Orca 해당 worktree에 이슈와 PR 전체 URL을 reference add로 연결하고 다시 조회해. 실제 worktree 선택자를 사용해.
STATE에 로컬 확인, 원격 저장, PR 병합, CI, 제품 실행 확인을 나눠 적어. 남은 차단은 필요한 사람의 행동과 함께 설명해.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] GitHub에서 실제 병합과 기본 브랜치 내용을 확인했다.
- [ ] 필수 CI의 최신 결과를 확인했고 없는 검사는 없음으로 적었다.
- [ ] 이슈와 PR의 상태가 실제 완료 범위와 일치한다.

막히면

CI가 실패하면 로그에서 원인을 찾아 현재 범위 안에서 수정합니다. 권한이나 사람의 리뷰가 필요한 경우 그 상태를 유지하고 필요한 행동을 알려줍니다.

다음 · D5 · 실제 전달·다음 재개 준비

D5 실제 전달과 다음 세션 준비를 마친다

개발 완료와 배포 완료를 구분하고 다음 시작점을 남깁니다.

열 창 개발 Codex 입력칸 + 실제 앱/배포 주소

세션 첫 출시 범위까지 같은 개발 세션 · 독립 다음 범위는 새 세션

1. 이 순서로 진행하세요

1. 로컬 실행만 요청했다면 로컬 확인으로 마칩니다. 배포가 범위에 있으면 기존 배포 경로와 대상 환경을 확인해 진행합니다. 새 서비스·비용·운영 구성을 임의로 추가하지 않습니다.
2. 배포 성공 메시지와 실제 서비스 동작을 구분합니다. 실제 URL에서 핵심 AC를 확인하고 어떤 버전이 실행 중인지 기록합니다.
3. STATE에 마지막 확인 커밋·미완료·다음 행동·관련 이슈/PR·작업자 상태를 남깁니다. 수정 중인 파일과 실행 프로세스가 남았는지도 확인합니다.
4. 작업자의 변경이 보존되고 더 쓸 채팅이 없다는 조건을 확인한 뒤 Orca가 안내하는 release/정리 절차를 따릅니다. 닫힌 채팅과 삭제된 브랜치를 같은 뜻으로 보지 않습니다.
5. 오늘 하던 일을 내일 이어가면 같은 개발 세션을 resume합니다. 첫 출시 범위를 끝내고 다른 기능을 시작하면 최신 문서를 읽는 새 세션을 만듭니다.

읽을 것

- 통합 검증·원격 상태·실제 실행

남길 것

- docs/STATE.md
- 전달 요약·필요 시 배포 증거

2. 표시된 입력 위치에 복사하세요

주 조정 Codex에 보낼 요청문

```
현재 단계의 에이전트 구성(역할·모델/effort·소유 파일·동시 인원·권한/비용)을 먼저 내게 검토받아. 승인된 범위에서 실행하고 협의·근거를 GitHub 이슈/Discussion에 저장해. 결과 revision과 검사·미검증, 다음 구성안을 보여줘. 수정 요청을 반영·재검증하고 내가 결과를 수용한 뒤 다음 단계로 넘어가.
이번에 승인된 범위의 최종 전달을 정리해줘.
로컬 실행·GitHub 저장·PR 병합·CI·배포·실제 URL 동작을 각각 완료/부분/미검증으로 표시하고 근거를 붙여.
배포가 이미 요청된 범위라면 기존 경로와 대상 환경을 확인해 진행하고 실제 서비스 동작까지 확인해. 요청되지 않았다면 로컬/원격 코드 전달 상태로 마쳐.
STATE를 최신화하고 다음 세션이 읽을 파일·첫 행동·남은 이슈·작업자 Run/Task/Dispatch를 기록해.
작업자와 worktree는 변경 보존·정리 가능 상태를 확인해 설치 가이드대로 처리해. 무응답을 종료로 간주하지 마.
내게는 실제 결과를 여는 방법과 남은 행동을 짧게 알려줘.
```

3. 실제로 확인한 항목만 체크하세요

- [ ] 내가 실행 결과 또는 배포 주소를 열어 확인했다.
- [ ] STATE만 보고 다음 행동을 알 수 있다.
- [ ] 활성·실패·종료 작업자가 명확히 구분된다.

> **막히면**
>
> 종료 전 기록을 못 했다면 다음 세션에서 현재 파일·Git·Orca 상태를 먼저 읽고 복구합니다. 기록 없이 완료를 추정하지 않습니다.

다음 · 완료 · 이어서 하면 resume / 독립 새 범위는 새 세션

개발 시작 시 문서 인계

P5 문서를 Gon OS의 개발 기준으로 이어갑니다

P5 이후, Gon OS의 개발 문서와 연결하기

P5의 PROJECT-BRIEF는 기획 인계 기준입니다. D1에서 개발 중 유지할 제품·구조 문서와 연결합니다. 기존 문서가 우선이며, 필요한 새 프로젝트에서는 PRODUCT.md가 목표·R/AC를, ARCHITECTURE.md가 확정 기술·데이터·구성·제약을 맡습니다. AEOS의 PROJECT.md·ARCHITECT.md가 있으면 그 경로를 유지합니다.

GitHub 연결 후 진행 상태의 기준은 Issues/기존 Projects입니다. TASKS·STATE는 실제 URL과 Orca ID를 담은 요약입니다. 로컬과 원격에 서로 다른 할 일 목록을 만들지 않습니다.

> **파일을 다시 쓰는 이유:** 기획에서 고른 범위를 개발 중 쓰는 제품·구조 문서에 반영하고 원문 결정과 연결하기 위해서입니다. 모든 파일에 같은 내용을 복제하지 않습니다. D1의 두 번째 요청문이 이 작업을 담당합니다.

읽고 따라 쓸 수 있는 자료

최종 개발 문서의 완성 모양과 빈 양식

예시는 “개인 할 일 앱을 학습용으로 만드는 경우”입니다. 실제로 수행한 조사·개발·승인 기록이 아닙니다. 내 프로젝트에는 빈 양식과 실제 답변을 사용하세요.

최종 문서 예시
(example-project/docs/PROJECT-BRIEF.md)

작업 계획 예시
(example-project/docs/TASKS.md)

인계 상태 예시
(example-project/docs/STATE.md)

최종 문서 빈 양식
(templates/PROJECT-BRIEF.md)

작업 계획 빈 양식
(templates/TASKS.md)

상태 빈 양식
(templates/STATE.md)

작업자 지시서 빈 양식
(templates/WORKER-SPEC.md)

P1~P5 모든 예시 읽기
(examples.html)

▼ 완성된 PROJECT-BRIEF 예시를 여기서 펼쳐 읽기

```
> 설명용 가상 예시입니다. 실제 사용자 조사·앱 구현·작동 검증을 수행한 결과가 아닙니다. 내 프로젝트에는 실제 답변·근거·날짜를 넣으세요.

# 개발 시작 문서 — 개인 할 일 앱 학습 예시

문서 버전: 예시 1 / 상태: 설명용 가상 합의 / 실제 승인일: 없음

실제 프로젝트에서는 사용자 확인 후 “개발 시작 승인”과 실제 날짜를 기록한다. 이 예시 파일만으로 실제 개발을 승인한 것으로 보지 않는다.

## 먼저 읽을 요약

개발 학습을 위해 같은 브라우저에서 할 일을 관리하는 작은 앱을 만든다. 첫 범위는 추가·완료/해제·삭제·새로고침 유지다. 로그인·서버·동기화는 제외한다. 첫 구현은 추가와 빈 입력 안내이며, 기술·저장 방식은 개발 준비에서 확인한다. 시작할 때 이 문서 → [TASKS](TASKS.md) → [STATE](STATE.md)를 읽는다.

## 사용자와 목표

개인 학습자 한 명. 실제 사용자가 다수라고 가정하지 않는다. 목적은 개발 과정과 확인 방법을 배우는 것이다. 근거는 [U01~U03](research/03-evidence.md), 선택 이유는 [D01~D03](research/04-decision.md)에 있다.

## 첫 범위와 완료 기준

| 요구 | 포함할 동작 | 직접 확인할 완료 기준 | 연결 결정 |
| --- | --- | --- | --- |
| R01 | 할 일 추가 | AC01: “우유 사기”를 넣고 추가하면 목록에 한 번 표시 | D02 |
| R02 | 완료·해제 | AC02: 완료 표시를 켜고 끌 수 있음 | D02 |
| R03 | 삭제 | AC03: 선택한 항목만 목록에서 사라짐 | D02 |
| R04 | 새로고침 유지 | AC04: 항목·완료 상태가 같은 브라우저 새로고침 후 일치 | D02·U02·U03 |
| R05 | 빈 입력 처리 | AC05: 공백만 넣으면 추가되지 않고 안내 표시 | D02 |

제외: 로그인, 사용자 계정, 여러 기기 동기화, 서버 API, 알림, 상용 배포. 브라우저 데이터 삭제 후 복구를 보장하지 않는다. 실제 저장 한계는
```

구현 전에 확인한다.

핵심 사용 흐름

앱 열기 → 할 일 입력 → 추가 → 완료 표시 → 새로고침 후 유지 확인 → 삭제. 목록이 비어 있을 때 안내도 확인한다.

데이터·기술·제약

데이터는 항목 식별자·내용·완료 여부다. 민감한 개인정보를 연습 데이터로 넣지 않는다. 기술과 저장 방식은 미정이다. 학습 예제 경로를 명시적으로 선택하면 React + TypeScript 프런트 구성을 후보로 삼을 수 있지만, 기존 프로젝트에 임의로 적용하지 않는다.

남은 미정과 시작 가능 범위

미정	담당·해결 시점	무엇을 막는가?
기술 선택	사용자와 개발 A, 환경 준비 전	환경 구성은 선택 후 진행
저장 API의 범위·삭제 조건	개발 A, 저장 구현 전	R04 구현·완료 판정은 확인 전 보류

이 문서로 새 세션의 인계 요약과 기술 비교는 시작할 수 있다. 미정을 확정한 척하여 전체 구현으로 넘어가지 않는다.

첫 작업과 다음 요청

첫 작업: R01·R05. 실제로 입력·추가·빈 입력을 확인한 뒤 R02~R04를 잇는다.

새 개발 세션에 보낼 요청:

“이 문서와 STATE를 실제로 읽고, 목표·포함·제외·AC번호·미정·첫 작업을 요약해줘. 설명용 예시 상태이므로 실제 프로젝트의 문서와 사용자 승인을 먼저 확인해. 기술이 미정이면 후보를 설명하고 내 선택을 기다려. 아직 구현하지 마.”

근거와 변경

[문제 정의](research/01-problem.md) → [조사 계획](research/02-research-plan.md) → [근거표](research/03-evidence.md) → [결정 기록](research/04-decision.md).

변경 이력: 예시 1 — 문답·선택을 R/AC번호로 연결한 교육용 문서 작성. 실제 프로젝트는 변경일·이유·수정 범위·현재 승인 상태를 남긴다.

Orca와 GitHub에 넘기는 방법

최종 문서와 작업 계획을 기준 커밋으로 저장한 다음 개발 작업자를 시작한다.
작업자에게는 이 문서, TASKS, 자신의 작업 지시서만 읽혀도 범위가 이해되어야 한다.
GitHub 이슈에는 관련 R/AC, PR에는 구현한 범위와 실제 검증을 연결한다.
현재 이 예시에는 실제 저장소·이슈·PR·Orca 실행 기록이 없다.

출발 질문	근거	결정	요구·완료 기준	작업
Q02/03 저장 범위·완료 기준	U02·U03, 저장 API는 미확인	D02	R04·AC04	K00 후 K03

출처가 미확인이라는 표시를 없애지 않고, 저장 구현 전에 확인해야 할 조건으로 전달한다.

Markdown(.md)은 글을 저장한 일반 텍스트 파일입니다. 앱이 열리지 않으면 “모든 예시 읽기”에서 브라우저로 확인할 수 있습니다.

막혔을 때

헷갈리는 상태를 이렇게 구분하세요

보이는 상황	의미	다음 행동
작업자를 시작했다 / ready	실행 준비 또는 시작 확인	완료 보고와 파일·검증을 기다림
주 조정자의 답변이 끝났다	Orca 알림 대기일 수 있음	주 조정 CLI를 닫지 않고 유지
worker_done이 왔다	작업자 보고 도착·Task 정리	보고한 성공/실패와 실제 증거 확인
작업자 무응답 / timeout	종료 여부 불명일 수 있음	worker-show/read와 recovery 안내로 상태 확인
Codex resume 성공	대화 기록을 다시 열었음	Run/Task·실제 파일은 별도로 대조
PR을 만들었다	변경 검토 요청이 존재함	최신 CI·리뷰·병합 상태 확인
CI가 초록색이다	자동 검사 범위가 통과함	실제 AC 동작·배포 버전 별도 확인

자동 재시도 제한에 걸리면 같은 작업을 새 Run/Task로 위장해 반복하지 않습니다. 실제 실패와 남은 자원을 확인해 사용자에게 필요한 선택을 알려야 합니다. 원격 작업자는 로컬과 후속 처리 조건이 다르므로 설치된 placement/recovery 가이드를 읽습니다.

▼ Orca 명령의 실제 형태 · AI가 참고할 부록

초보자는 실행 단계의 자연어 요청문을 사용하면 됩니다. 아래는 주 조정자가 설치 가이드를 읽고 실제 ID를 대체해 사용할 참고입니다. 그대로 복사 실행할 명령 묶음이 아닙니다.

```
# 아래는 AI가 실제 ID·브랜치를 채워 사용할 참고 형식입니다.
# 이 상자는 그대로 실행하지 않습니다. 중괄호 값은 실제 조회 결과로 대체합니다.

# 조사: 같은 작업공간, 분리된 파일. --spec은 필요 시 Run을 자동 생성합니다.
orca-ide terminal create --worktree current --title research-a --command 'codex --yolo' --json
orca-ide terminal wait --terminal {TERMINAL_HANDLE} --for tui-idle --timeout-ms 60000 --json
orca-ide orchestration worker-start --spec-file {지시서_절대경로} --worktree current --terminal {TERMINAL_HANDLE} --json

# 선행 관계가 있는 계획: Run 생성 후 Task와 의존성 기록
orca-ide orchestration run-create --objective "첫 출시 개발" --json
orca-ide orchestration task-create --spec-file docs/tasks/base.md --json
orca-ide orchestration task-create --spec-file docs/tasks/feature.md --deps '["{실제_선행_TASK_ID}"]' --json
orca-ide orchestration task-list --ready --brief

# 코드 작업: 실제 저장소·기준 브랜치에서 새 worktree
orca-ide worktree create --repo id:{REPO_ID} --name feature-one --base-branch {기준_브랜치} --json
orca-ide terminal create --worktree {WORKTREE_SELECTOR} --title feature-one --command 'codex --yolo' --json
orca-ide terminal wait --terminal {TERMINAL_HANDLE} --for tui-idle --timeout-ms 60000 --json
orca-ide orchestration worker-start --task {TASK_ID} --worktree {WORKTREE_SELECTOR} --terminal {TERMINAL_HANDLE} --json

# 추가 지시 / 결과 확인
orca-ide orchestration send --to dispatch:{DISPATCH_ID} --subject "검증 보완" --body "AC04의 새로고침 확인 결과를 추가해줘" --json
orca-ide orchestration worker-read --dispatch {DISPATCH_ID} --source auto --limit 80 --json
```

```
# 질문에 답하기 (사용자의 기존 범위 안에서)
orca-ide orchestration reply --id {MESSAGE_ID} --body "확정된 답변" --json

# 재개 시 상태 확인. 실제 소유권 확인 전 run-use를 무조건 실행하지 않음.
orca-ide orchestration run-list --json
orca-ide orchestration worker-list --run {RUN_ID} --include-remote --json
orca-ide orchestration run-use --id {RUN_ID} --json

# GitHub 전체 URL을 실제 작업공간에 연결
orca-ide reference add {ISSUE_URL} {PR_URL} --worktree {WORKTREE_SELECTOR} --json
```

이 가이드의 직접 명령은 확인한 Linux 실행 파일 `orca-ide` 기준입니다. Orca 관리 환경의 AI는 ORCA_CLI_COMMAND 등 설치 스킬의 선택 규칙으로 실행 파일을 정하고 계속 같은 것을 사용합니다. CLI별 승인 생략 옵션으로 연 터미널을 worker-start --terminal로 연결합니다. 터미널 생성 응답의 handle과 worktree 생성 응답의 실제 selector를 사용합니다. timeout·연결 실패 시 기존 handle을 기록하고 상태를 먼저 확인합니다. 재시도도 같은 Task와 배치에 승인 생략 CLI 터미널을 연결하고, 복구 가이드의 liveness·next·release 조건을 따릅니다. worker-start 실행 후에는 응답을 끝내 알림을 기다리고, wake에 포함된 check와 반환된 next를 따릅니다.

▼ 용어를 일상적인 말로 이해하기

용어	쉽게 말하면	주의할 차이
Codex 세션	AI와 이어지는 한 대화	기획과 개발을 구분하는 단위
Orca 작업공간	폴더 또는 Git checkout과 탭이 모인 자리	같은 폴더의 여러 AI는 파일을 공유함
Git worktree	같은 저장소를 다른 폴더에서 다루는 별도 작업 사본	병렬 코드 수정 충돌을 줄이나 통합은 별도 필요
Run	이번 목표를 묶는 작업판	예: 첫 출시 개발
Task	목표 안의 작업 항목	예: 저장 기능 구현
Dispatch	Task를 실제 작업자에게 맡긴 실행 1회	재시도는 같은 Task의 다른 실행이 될 수 있음
GitHub Issue	해결할 문제와 완료 기준의 원격 기록	Orca Task ID와 다른 번호
PR / CI	변경 검토·병합 요청 / 서버의 자동 검사	실제 제품 사용 확인과 구분

확인 범위와 근거

이 안내서가 확인한 것

설치된 Orca 1.4.246의 프로젝트 추가·에이전트 설정 화면 문구와 orca-cli·orchestration 가이드, coordinator-loop·placement·recovery 참고, agent launch·worker-start·task-create·run-use와 terminal create/wait·worktree create 도움말과 Codex CLI 0.166.0의 resume/fork 도움말을 읽어 작성했습니다. 화면 메뉴명은 설치 리소스에서 확인했으며 GUI 클릭·저장 동작 자체는 시험하지 않았습니다. 실제 사용자의 프로젝트에 작업자를 실행하거나 GitHub 저장소를 만들고 병합한 결과는 아닙니다.

- OpenAI · Codex 명령과 세션
- GitHub CLI · 저장소 생성 · 브라우저 인증
- GitHub CLI · PR 생성 · PR 자동 검사 확인
- GitHub · PR과 이슈 연결·자동 종료 조건

Orca는 설치 버전의 스킬 가이드를 기준으로 합니다. 업데이트 후 명령이 달라졌다면 AI에게 현재 가이드를 다시 읽게 하세요. 이 안내서의 “같은 기획 → 새 개발”과 파일 이름은 이해를 돕는 권장 운영 방식이며 제품의 강제 규칙이 아닙니다.

기존 Herdr 안내서는 별도로 보존되어 있습니다. 이 판은 Orca 오케스트레이션 기준입니다.